NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

Introduction To Python Programming

Course Outcomes:

At the end of the course student will be able to

1. Understand and experiment with the basics of python programming like data types and looping.
2. Describe the concept of functions in python for solving real-world problems.
3. Apply the Python operations for manipulating strings, lists, tuples and dictionaries.
4. Demonstrate the proficiency in handling File Systems.
5. Understand the basic knowledge on Exception handling and plotting graphs.

Textbooks:

1. Kenneth A. Lambert, “The Fundamentals of Python: First Programs”, Cengage Learning, 2nd Edition, 2019.
2. Yashwanth Kanetkar, Adithya Kanetkar, “Let us Python”, 3rd Edition.
3. Mark Summerfield, “Programming in Python 3 - A Complete Introduction to the Python Language”, Second Edition, Addison-Wesley, 2009.
4. Y. Daniel Liang, “Introduction to Programming Using Python”, Pearson, 2013.

Tuples , Sets, Dictionaries

Tuples

- A tuple is same as list, except that the set of elements is enclosed in parentheses instead of square brackets.
- A tuple is an **immutable** list. i.e. once a tuple has been created, you can't add elements to a tuple or remove elements from the tuple.
- But tuple can be converted into list and list can be converted into tuple.

methods	example	description

list( )	>>> a=(1,2,3,4,5)
	>>> a=list(a)
	>>> print(a)
	[1, 2, 3, 4, 5]
		it convert the given tuple into list.

tuple( )	>>> a=[1,2,3,4,5]
	>>> a=tuple(a)
	>>> print(a)
	(1, 2, 3, 4, 5)
		it convert the given list into tuple.

Benefit of Tuple:

- Tuples are faster than lists.
- If the user wants to protect the data from accidental changes, tuple can be used.
- Tuples can be used as keys in dictionaries, while lists can't.

Operations on Tuples:

1. Indexing
2. Slicing
3. Concatenation
4. Repetitions
5. Membership
6. Comparison

Operations	examples	description
Creating a tuple	>>>a=(20,40,60,"apple","ball")	Creating the tuple with elements of different data types.
Indexing	>>>print(a[0]) 20 >>> a[2] 60	Accessing the item in the position 0 Accessing the item in the position 2
Slicing	>>>print(a[1:3]) (40,60)	Displaying items from 1st till 2nd.
Concatenation	>>> b=(2,4) >>>print(a+b) >>>(20,40,60,"apple","ball",2,4)	Adding tuple elements at the end of another tuple elements
Repetition	>>>print(b*2) >>>(2,4,2,4)	repeating the tuple in n no of times

Membership	>>> a=(2,3,4,5,6,7,8,9,10)
	>>> 5 in a
	True
	>>> 100 in a
	False
	>>> 2 not in a
	False
		Returns True if element is present in tuple. Otherwise returns false.

Comparison	>>> a=(2,3,4,5,6,7,8,9,10)
	>>>b=(2,3,4)
	>>> a==b
	False
	>>> a!=b
	True
		Returns True if all elements in both elements are same. Otherwise returns false

Tuple methods:

- Tuple is immutable so changes cannot be done on the elements of a tuple once it is assigned.

methods	example	description

a.index(tuple)	>>> a=(1,2,3,4,5)
	>>> a.index(5)
	4
		Returns the index of the first matched item.

a.count(tuple)	>>>a=(1,2,3,4,5) >>> a.count(3) 1	Returns the count of the given element.
len(tuple)	>>> len(a) 5	return the length of the tuple
min(tuple)	>>> min(a) 1	return the minimum element in a tuple
max(tuple)	>>> max(a) 5	return the maximum element in a tuple
del(tuple)	>>> del(a)	Delete the entire tuple.

Tuple Assignment:

- Tuple assignment allows, variables on the left of an assignment operator and values of tuple on the right of the assignment operator.
- Multiple assignment works by creating a tuple of expressions from the right hand side, and a tuple of targets from the left, and then matching each expression to a target.
- Because multiple assignments use tuples to work, it is often termed tuple assignment.

Uses of Tuple assignment:

- It is often useful to swap the values of two variables.
- Example:
- Swapping using temporary variable:

```
a=20
b=50
temp = a
a = b
b = temp
print("value after swapping is",a,b)
```

Swapping using tuple assignment:

```
a=20
b=50
(a,b)=(b,a)
print("value after swapping is",a,b)
```

Multiple assignments:

- Multiple values can be assigned to multiple variables using tuple assignment.

```
>>>(a,b,c)=(1,2,3)
>>>print(a)
1
>>>print(b)
2
>>>print(c)
3
```

Tuple as return value:

- A Tuple is a comma separated sequence of items.
- It is created with or without ( ).
- A function can return one value. if you want to return more than one value from a function. we can use tuple as return value.

Eg:

```
def div(a,b):
  r=a%b
  q=a//b
  return(r,q)
a=int(input("enter a value:"))
b=int(input("enter b value:"))
r,q=div(a,b)
print("reminder:",r)
print("quotient:",q)
```

```
Output:
enter a value:4
enter b value:3
reminder: 1
quotient: 1
```

Eg:2

```
def minmax(a):
    small=min(a)
    big=max(a)
    return(small, big)
a=[1,2,3,4,6]
small, big=minmax(a)
print("smallest:", small)
print("biggest:", big)
```

Output:

smallest: 1

biggest: 6

Sets

Sets

- A Set in Python is used to store a collection of items with the following properties.
- 1)No duplicate elements. If try to insert the same item again, it overwrites previous one.

2)An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.

Unchangeable

Set items are unchangeable, meaning that we cannot change the items after the set has been created.

```
s = {10, 50, 20}
print(s)
print(type(s))
```

Output

```
{10, 50, 20}
<class 'set'>
```

Type Casting with Python Set method

The Python set() method is used for type casting.

```python
# typecasting list to set
s = set(["a", "b", "c"])
print(s)

# Adding element to the set
s.add("d")
print(s)
```

Output

```
{'c', 'b', 'a'}
{'d', 'c', 'b', 'a'}
```

Accessing Set Elements

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
  print(x)
```

Since sets are unordered, output order may change.

```
apple
banana
cherry
```

Python – Add Set Items

- Add an item to a set, using the add() method:

Add an item to a set, using the add() method:

```
thisset = {"apple", "banana", "cherry"}

thisset.add("orange")

print(thisset)
```

Add Sets

Example

Add elements from `tropical` into `thisset`:

```
thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango", "papaya"}

thisset.update(tropical)

print(thisset)
```

- add() → one item
- update() → many items

O/P:{'apple', 'banana', 'cherry', 'pineapple', 'mango', 'papaya'}

Add Any Iterable

- The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

Example

Add elements of a list to at set:

```
thisset = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]

thisset.update(mylist)

print(thisset)
```

{'apple', 'banana', 'cherry', 'kiwi', 'orange’}
Because update() only needs something it can **iterate over**.

Python – Remove Set Items

Remove Item

- To remove an item in a set, use the remove(), or the discard() method.(If the item to remove does not exist, remove() will raise an error)

Example

Remove "banana" by using the `remove()` method:

```
thisset = {"apple", "banana", "cherry"}

thisset.remove("banana")

print(thisset)
```

remove() → gives error if item not found
discard() → no error if item not found

discard() method

- Remove "banana" by using the discard() method:If the item to remove does not exist, discard() will NOT raise an error.

```
thisset = {"apple", "banana", "cherry"}

thisset.discard("banana")

print(thisset)
```

pop() method :

- You can also use the pop() method to remove an item, but this method will remove a random item, so you cannot be sure what item that gets removed.
- The return value of the pop() method is the removed item.

Remove a random item by using the `pop()` method:

```
thisset = {"apple", "banana", "cherry"}

x = thisset.pop()

print(x)

print(thisset)
```

clear():

- The clear() method empties the set:
- thisset.clear()

Example:

thisset = {"apple", "banana", "cherry"}

thisset.clear()

print(thisset) O/p: set()

The del keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}

del thisset

print(thisset)
```

Join Sets

- There are several ways to join two or more sets in Python.
- The union() and update() methods joins all items from both sets.
- The intersection() method keeps ONLY the duplicates.
- The difference() method keeps the items from the first set that are not in the other set(s).
- The symmetric_difference() method keeps all items EXCEPT the duplicates.

- **1. union() — Combine all items**

Joins all unique items from both sets
Duplicates are removed automatically
Returns a **new set**

A = {"apple", "banana", "cherry"}
B = {"banana", "mango", "grapes"}

Example:

C = A.union(B)

print(C)
O/p:
{'apple', 'banana', 'cherry', 'mango', 'grapes’}

All items included
No duplicates

2. update() — Add items into original set

Same result as union
BUT modifies the original set
Does NOT create a new set

Example:

```
A.update(B)
print(A)
```

Output

- {'apple', 'banana', 'cherry', 'mango', 'grapes'}

3. intersection() — Keep only common items

Keeps **only duplicates (common elements)**
Removes all others

Example

```
C = A.intersection(B)
print(C)
```

Output

```
{'banana'}
```

Only common item remains

4. difference() — Keep items only in first set

Keeps items in first set
Removes items that are in second se

Example

```
C = A.difference(B)
print(C)
```

Output

{'apple', 'cherry’}

Items unique to A

5. symmetric_difference() — Keep non-common items

Removes duplicates
Keeps items that are **not common**

Think: **union − intersection**

Example

```
C = A.symmetric_difference(B)
print(C)
```

Output

{'apple', 'cherry', 'mango', 'grapes'}

Everything except the common item

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set3 = set1.union(set2)
print(set3)
myset = set1.union(set2, set3, set4)
```

Use | to join two sets:

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set3 = set1 | set2
print(set3)
myset = set1 | set2 | set3 |set4
```

Dictionaries

- Dictionary is an unordered collection of elements.
- An element in dictionary has a key: value pair.
- All elements in dictionary are placed inside the curly braces i.e. { }
- Elements in Dictionaries are **accessed via keys** and not by their position.
- The values of a dictionary can be any data type.
- Keys must be immutable data type (numbers, strings, tuple)

Operations on dictionary:

1. Accessing an element
2. Update
3. Add element
4. Membership

Operations	Example	Description

Creating a dictionary	>>> a={1:"one",2:"two"}
	>>> print(a)
	{1: 'one', 2: 'two'}
		Creating the dictionary with elements of different data types.

accessing an element	>>> a[1]
	'one'
	>>> a[0]
	KeyError: 0
		Accessing the elements by using keys.

Update	>>> a[1]="ONE"
	>>> print(a)
	{1: 'ONE', 2: 'two'}
		Assigning a new value to key. It replaces the old value by new value.

add element	>>> a[3]="three"
	>>> print(a)
	{1: 'ONE', 2: 'two', 3: 'three'}
		Add new element in to the dictionary with key.

membership	a={1: 'ONE', 2: 'two', 3: 'three'}
	>>> 1 in a
	True
	>>> 3 not in a
	False
		Returns True if the key is present in dictionary. Otherwise returns false.

a.update(dictionary)	>>> b={4:"four"}
	>>> a.update(b)
	>>> print(a)
	{1: 'ONE', 2: 'two', 3: 'three', 4: 'four'}
		It will add the dictionary with the existing dictionary

fromkeys()	>>> key={"apple","ball"}
	>>> value="for kids"
	>>> d=dict.fromkeys(key,value)
	>>> print(d)
	{'apple': 'for kids', 'ball': 'for kids'}
		It creates a dictionary from key and values.

len(a)	a={1: 'ONE', 2: 'two', 3: 'three'} >>>lena(a) 3	It returns the length of the list.
clear()	a={1: 'ONE', 2: 'two', 3: 'three'} >>>a.clear() >>>print(a) >>>{}	Remove all elements form the dictionary.
del(a)	a={1: 'ONE', 2: 'two', 3: 'three'} >>> del(a)	It will delete the entire dictionary.

Methods in dictionary:

Method	Example	Description

a.copy( )	a={1: 'ONE', 2: 'two', 3: 'three'}
	>>> b=a.copy()
	>>> print(b)
	{1: 'ONE', 2: 'two', 3: 'three'}
		It returns copy of the dictionary. here copy of dictionary 'a' get stored in to dictionary 'b'

a.items()	>>> a.items() dict_items([(1, 'ONE'), (2, 'two'), (3, 'three')])	Return a new view of the dictionary's items. It displays a list of dictionary's (key, value) tuple pairs.
a.keys()	>>> a.keys() dict_keys([1, 2, 3])	It displays list of keys in a dictionary
a.values()	>>> a.values() dict_values(['ONE', 'two', 'three'])	It displays list of values in dictionary

a.pop(key)	>>> a.pop(3)
	'three'
	>>> print(a)
	{1: 'ONE', 2: 'two'}
		Remove the element with *key* and return its value from the dictionary.

setdefault(key,value)	>>> a.setdefault(3,"three")
	'three'
	>>> print(a)
	{1: 'ONE', 2: 'two', 3: 'three'}
	>>> a.setdefault(2)
	'two'
		If key is in the dictionary, return its value. If key is not present, insert key with a value of dictionary and return dictionary.

Difference between List, Tuples and dictionary:

List	Tuples	Dictionary
A list is mutable	A tuple is immutable	A dictionary is mutable
Lists are dynamic	Tuples are fixed size in nature	In values can be of any data type and can repeat, keys must be of immutable type
List are enclosed in brackets[ ] and their elements and size can be changed	Tuples are enclosed in parenthesis ( ) and cannot be updated	Tuples are enclosed in curly braces { } and consist of key:value
Example: List = [10, 12, 15]	Example: Words = ("spam", "egss") Or Words = "spam", "eggs"	Example: Dict = {"ram": 26, "abi": 24}
Access: print(list[0])	Access: print(words[0])	Access: print(dict["ram"])
Can contain duplicate elements	Can contain duplicate elements. Faster compared to lists	Cant contain duplicate keys, but can contain duplicate values
Slicing can be done	Slicing can be done	Slicing can't be done
Usage: ❖ List is used if a collection of data that doesnt need random access. ❖ List is used when data can be modified frequently	Usage: ❖ Tuple can be used when data cannot be changed. ❖ A tuple is used in combination with a dictionary i.e.a tuple might represent a key.	Usage: ❖ Dictionary is used when a logical association between key:value pair. ❖ When in need of fast lookup for data, based on a custom key. ❖ Dictionary is used when data is being constantly modified.